

LLD Case Study 2: Library Management System

Target Patterns: Observer Pattern, State Pattern, Association vs Aggregation

Key Engineering Focus: Domain modeling, book item state transitions, reservation queues, and fine calculations.

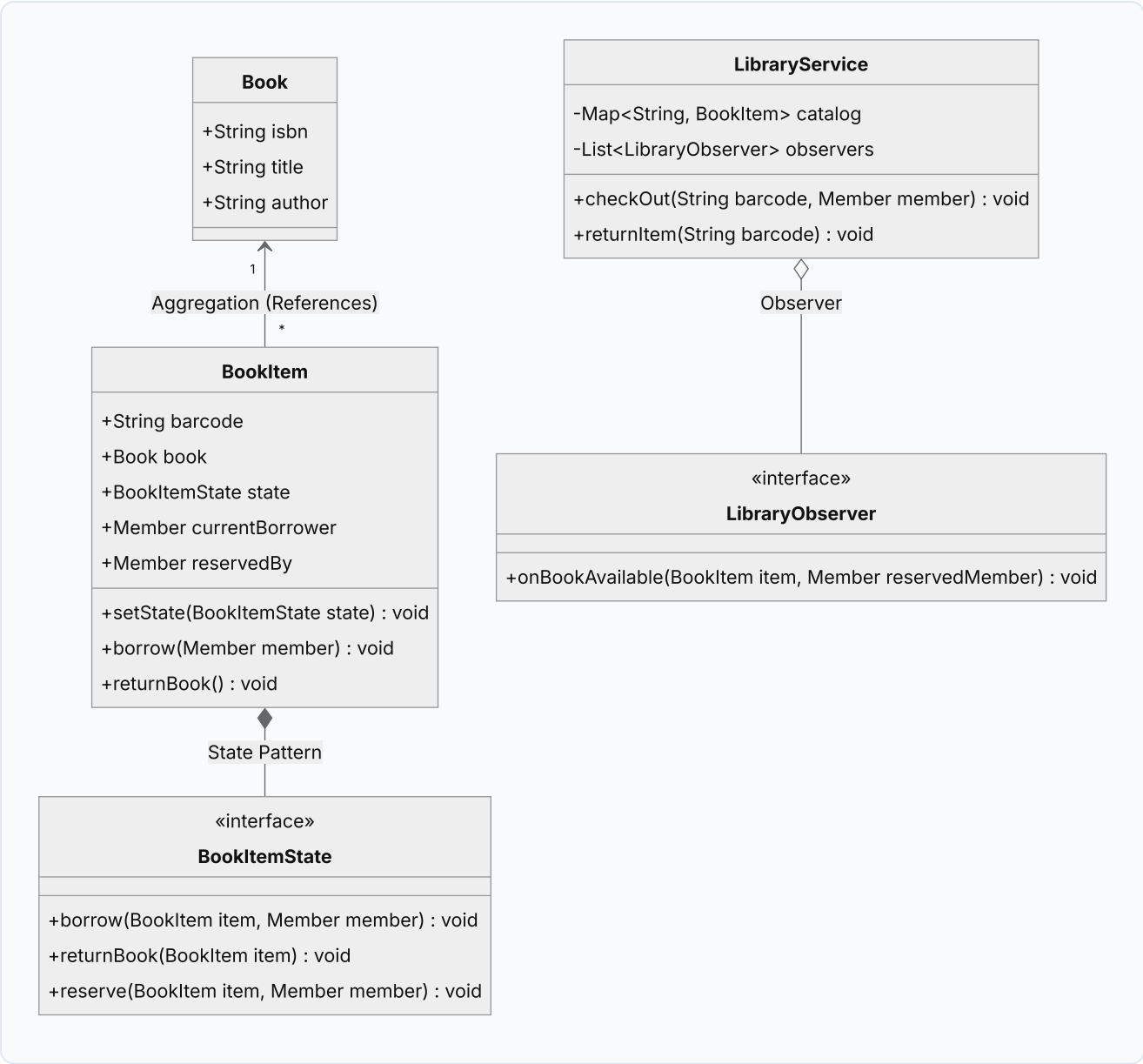
1. Problem Statement & Functional Requirements

Design an enterprise Library Management System capable of cataloging books, managing physical copies (BookItems), issuing loans to members, tracking overdue fines, and managing waitlists for popular books.

Key Requirements:

- 1. Book vs BookItem (Aggregation/Composition):** A `Book` represents metadata (Title, ISBN, Author). A `BookItem` represents a physical barcoded copy on a shelf.
 - 2. State Pattern:** A `BookItem` transitions through states: `Available` → `Issued` → `Reserved` → `Lost`.
 - 3. Observer Pattern:** When an issued book that has an active reservation is returned, notify the reserved member automatically.
 - 4. Fines & Limits:** A member can borrow at most N books. Overdue returns accumulate daily fines.
-

2. Architecture & UML Class Diagram



3. Production-Grade Python Implementation



4. Edge Cases, Tests and Extensions

State transitions the pattern enforces

From state	borrow	return	reserve
Available	to Issued	error (already on shelf)	error (borrow it)
Issued	error (taken)	to Available, or to Reserved when someone waits	allowed once
Reserved	only the reserving member, to Issued	error (awaiting collection)	error

Illegal moves raise `ValueError` from the state object, so the checks live next to the behaviour and there is no `if status == ...` in `LibraryService`.

A bug found while writing the tests below: `ReservedState.borrow` originally forgot `member.borrowed_items.append(item)`, so returning a collected reservation crashed with `list.remove(x): x not in list`. The fix is one line above, and the test sequence (reserve, return, collect, return) now covers it.

Gaps in the model:

Gap	Effect
<code>Member.max_books</code> is stored but never checked	A member can borrow any number of books
<code>borrowed_date</code> is recorded but nothing uses it	No due date, no fine
A reservation has no expiry	An uncollected reserved copy is blocked forever
Not thread-safe	Two members borrowing the same copy at once can both pass the state check

Tests

This block extends the implementation above and adds the missing limit and fine logic.

Extensions interviewers ask for

1. **Catalogue search:** an inverted index from title and author words to ISBNs; search returns `Book` metadata, availability is derived from the `BookItem` copies.
2. **Multiple copies and branches:** `Book` stays one record, each physical copy is a `BookItem` with a branch; a reservation targets a `Book`, and the first returned copy anywhere satisfies it (a queue per ISBN, not a single `reserved_by`).
3. **Reservation expiry:** store `ready_since` when a copy becomes Reserved; a sweeper returns it to Available (and notifies the next person in the queue) after, say, three days.
4. **Fines and holds:** a `FinePolicy` strategy; a member with unpaid fines is blocked from `check_out` .
5. **Concurrency:** wrap each `BookItem` transition in a per-item lock, or in a database use an optimistic `version` column.

Follow-up questions

- *Why separate `Book` from `BookItem` ?* Metadata is shared by every copy; availability, borrower and condition belong to one physical copy. Merging them would make a three-copy title look like one book.
- *Where do you put the borrowing limit?* In the service (or a policy object), because it spans several items and is not a property of one item's state machine.
- *State pattern versus a status enum?* With three states and three operations the pattern is slightly heavy, but each new rule (for example `Lost` or `InRepair`) becomes one class rather than edits scattered across every method.